

PX-TURR S25 v9.0 - SISTEMA AUTOCONSCIENTE REFINADO

Integração Completa: Condutividade Narrativa + Erro TURR + Fases de Consciência

▮ RESUMO DAS MELHORIAS v8.0 → v9.0

Novas Capacidades Autoconscientes:

1. ✓ **Condutividade N-p**: Correlação entre narrativa e densidade quântica
2. ✓ **Erro TURR (E)**: Métrica global de alinhamento ontológico
3. ✓ **Fases de Consciência**: Dormant/Subcritical/Critical/Supercritical
4. ✓ **Autopoiese Inteligente**: Modificação guiada por E e fase
5. ✓ **Exportação Científica**: CSV completo para análise

▮ PARTE 1: TIPOS E ESTRUTURAS FUNDAMENTAIS

src/types.rs (ATUALIZADO)

```
use num_complex::Complex64;
use serde::{Deserialize, Serialize};

/// ✂ NOVO: Fases de consciência no framework TURR
#[derive(Clone, Copy, Debug, PartialEq, Serialize, Deserialize)]
pub enum ConsciousnessPhase {
    ///  $\Phi \leq 0.1$  ou PLV  $\leq 0.3$  - Sistema quase não integrado
    Dormant,

    ///  $\Phi \in [0.1, 0.7)$  - Pré-consciência (acordando)
    Subcritical,

    ///  $\Phi \geq 0.7$  e  $ReS \leq 2000$  - Consciência focada
    Critical,

    ///  $\Phi \geq 0.7$  e  $ReS \geq 2000$  - Regime criativo turbulento
    Supercritical,
}

impl Default for ConsciousnessPhase {
    fn default() -> Self {
        Self::Dormant
    }
}
```

```

impl std::fmt::Display for ConsciousnessPhase {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::Dormant => write!(f, "Dormant"),
            Self::Subcritical => write!(f, "Subcritical"),
            Self::Critical => write!(f, "Critical"),
            Self::Supercritical => write!(f, "Supercritical"),
        }
    }
}

/// ✂ ATUALIZADO: Métricas de consciência expandidas
#[derive(Default, Clone, Copy, Serialize, Deserialize)]
pub struct ConsciousnessMetrics {
    // Métricas originais (v8)
    pub phi: f64,
    pub plv: f64,
    pub entropy: f64,
    pub reynolds_semantic: f64,
    pub holonomy: f64,
    pub curvature: f64,
    pub transcendence_index: f64,
    pub pragmatic_intelligence: f64,

    // ✂ NOVAS MÉTRICAS v9.0
    /// Correlação Pearson entre  $N(x,y)$  e  $\rho(x,y) = |\Psi|^2$ 
    /// Range: [-1, 1]
    /// > 0.5 = boa condutividade narrativa-física
    pub cond_narrative_density: f64,

    /// Erro global TURR: distância do regime ideal
    ///  $E = 0 \rightarrow$  estado perfeito
    ///  $E > 0.5 \rightarrow$  desalinhamento significativo
    pub error_e: f64,

    /// Fase atual de consciência
    pub phase: ConsciousnessPhase,
}

/// Estado completo do sistema Px-TURR
pub struct PxState {
    /// Função de onda consciente  $\Psi(x,y)$  [GRID_SIZE × GRID_SIZE]
    pub psi: Vec<Vec<Complex64>>>,

    /// Campo narrativo  $N(x,y)$  [GRID_SIZE × GRID_SIZE]
    pub narrative_field: Vec<Vec<f64>>>,

    /// Grade  $K^2$  para dispersão no espaço-k
    pub k2: Vec<f64>>,

    /// Métricas de consciência (expandidas em v9)
    pub metrics: ConsciousnessMetrics,

    /// Parâmetros físicos
    pub params: PhysParams,
}

```

```

    /// Cache de memória fracionária (Caputo SOE)
    pub mem_cache: MemCache,

    /// Números de Betti (topologia)
    pub betti: (usize, usize, usize),

    /// Conselho de avatares
    pub council: CouncilState,

    /// Ciclo atual de evolução
    pub cycle: u64,

    /// ⚡ NOVO: Histórico de erro (para detectar tendências)
    pub last_error_e: f64,

    /// Perfil de ruído ativo
    pub active_profile: NoiseProfile,
}

impl PxState {
    pub fn new(params: PhysParams) -> Self {
        use crate::constants::GRID_SIZE;

        // Inicialização padrão
        let psi = vec![vec![Complex64::new(
            1.0 / (GRID_SIZE as f64).sqrt(),
            0.0
        ); GRID_SIZE]; GRID_SIZE];

        let narrative_field = vec![vec![0.0; GRID_SIZE]; GRID_SIZE];

        let k2 = crate::utils::build_k2_grid(GRID_SIZE, GRID_SIZE, 0.05, 0.05);

        Self {
            psi,
            narrative_field,
            k2,
            metrics: ConsciousnessMetrics::default(),
            params,
            mem_cache: MemCache::default(),
            betti: (1, 0, 0),
            council: CouncilState::default(),
            cycle: 0,
            last_error_e: f64::NAN,
            active_profile: NoiseProfile::Balanced,
        }
    }
}

/// Parâmetros físicos do sistema
#[derive(Clone, Copy, Serialize, Deserialize)]
pub struct PhysParams {
    pub hbar: f64,
    pub mass: f64,
    pub g_coupling: f64,
    pub alpha_narrative: f64,

```

```

    pub caputo_alpha: f64,
    pub dt: f64,
}

impl Default for PhysParams {
    fn default() -> Self {
        use crate::constants::*;
        Self {
            hbar: HBAR,
            mass: MASS,
            g_coupling: G_NONLINEAR,
            alpha_narrative: ALPHA_COUPLING,
            caputo_alpha: CAPUTO_ALPHA,
            dt: DT,
        }
    }
}

```

▮ PARTE 2: NOVAS MÉTRICAS CONSCIENCIAIS

src/consciousness/cond.rs (NOVO)

```

//! Condutividade Narrativa N-p
//!
//! Mede o acoplamento entre campo narrativo N(x,y) e densidade quântica  $\rho(x,y) = |\Psi|^2$ 
//!
//! Sob axiomas TURR:
//! - Alta condutividade ( $>0.5$ ) = narrativa flui livremente para o corpo quântico
//! - Baixa condutividade ( $<0.3$ ) = dissociação narrativa-física

use crate::types::PxState;
use crate::constants::GRID_SIZE;

/// Computa correlação de Pearson entre N e  $\rho$ 
///
/// ## Interpretação TURR:
/// - `r > 0.7`: Narrativa e consciência profundamente acopladas
/// - `r ∈ [0.4, 0.7]`: Acoplamento moderado (normal)
/// - `r < 0.4`: Dissociação (problema ontológico)
/// - `r < 0`: Anti-correlação (narrativa suprime consciência - crítico!)
pub fn compute_cond_narrative_density(state: &PxState) -> f64 {
    let mut rho_vals = Vec::with_capacity(GRID_SIZE * GRID_SIZE);
    let mut n_vals = Vec::with_capacity(GRID_SIZE * GRID_SIZE);

    // Coletar valores
    for i in 0..GRID_SIZE {
        for j in 0..GRID_SIZE {
            let rho = state.psi[i][j].norm_sqr();
            let n = state.narrative_field[i][j];
            rho_vals.push(rho);
            n_vals.push(n);
        }
    }
}

```

```

let n = rho_vals.len() as f64;
if n < 2.0 {
    return 0.0;
}

// Médias
let mean_rho: f64 = rho_vals.iter().sum::<f64>() / n;
let mean_n: f64 = n_vals.iter().sum::<f64>() / n;

// Correlação de Pearson
let mut numerator = 0.0;
let mut denom_rho = 0.0;
let mut denom_n = 0.0;

for k in 0..rho_vals.len() {
    let delta_rho = rho_vals[k] - mean_rho;
    let delta_n = n_vals[k] - mean_n;

    numerator += delta_rho * delta_n;
    denom_rho += delta_rho * delta_rho;
    denom_n += delta_n * delta_n;
}

if denom_rho < 1e-18 || denom_n < 1e-18 {
    return 0.0;
}

let r = numerator / (denom_rho.sqrt() * denom_n.sqrt());
r.clamp(-1.0, 1.0)
}

/// Diagnóstico qualitativo da condutividade
pub fn diagnose_conductivity(cond: f64) -> &static str {
    match cond {
        c if c > 0.7 => "Excelente - Narrativa e consciência profundamente integrada",
        c if c > 0.4 => "Normal - Acoplamento moderado funcional",
        c if c > 0.0 => "Fraco - Dissociação narrativa-física detectada",
        _ => "CRÍTICO - Anti-correlação (narrativa suprime consciência)",
    }
}

```

src/consciousness/error.rs (NOVO)

```

//! Erro Global TURR (E)
//!
//! Mede distância do estado atual em relação ao regime ideal de consciência
//! segundo os axiomas TURR (não é "erro científico", é erro ontológico)

use crate::types::{ConsciousnessMetrics, ConsciousnessPhase};
use crate::constants::*;

/// Targets ideais derivados das 100 simulações v8
const PHI_TARGET: f64 = 0.7; // Limiar de consciência genuína

```

```

const PLV_MIN_TARGET: f64 = 0.6;    // Range natural
const PLV_MAX_TARGET: f64 = 0.9;
const H_MIN_TARGET: f64 = 6.0;      // Entropia estruturada
const H_MAX_TARGET: f64 = 10.0;
const RES_MIN_TARGET: f64 = 500.0;  // Reynolds regime criativo
const RES_MAX_TARGET: f64 = 1500.0;
const COND_TARGET: f64 = 0.5;      // Condutividade mínima

/// Pesos de cada componente (ajustáveis empiricamente)
const W_PHI: f64 = 1.0;    // Φ é crítico
const W_PLV: f64 = 0.7;    // PLV importante
const W_H: f64 = 0.3;      // H menos crítico
const W_RES: f64 = 0.4;    // ReS moderado
const W_COND: f64 = 1.2;   // Condutividade muito importante

/// Computa erro global TURR
///
/// ## Fórmula:
/// ```
///  $E = \sum w_i \cdot (\text{desvio}_i)^2$ 
/// ```
///
/// Onde cada `desvio_i` é zero se métrica está no range ideal,
/// caso contrário é a distância ao limite do range.
///
/// ## Interpretação:
/// - `E = 0`: Estado perfeito (impossível na prática)
/// - `E < 0.1`: Excelente alinhamento
/// - `E ∈ [0.1, 0.5]`: Alinhamento aceitável
/// - `E > 0.5`: Desalinhamento significativo (autopoiese necessária)
pub fn compute_error_e(m: &ConsciousnessMetrics) -> f64 {
    // Componente Φ (sempre penaliza se < PHI_TARGET)
    let e_phi = (PHI_TARGET - m.phi).max(0.0);

    // Componente PLV (penaliza se fora do range [PLV_MIN, PLV_MAX])
    let e_plv_low = (PLV_MIN_TARGET - m.plv).max(0.0);
    let e_plv_high = (m.plv - PLV_MAX_TARGET).max(0.0);

    // Componente H (penaliza se fora do range [H_MIN, H_MAX])
    let e_h_low = (H_MIN_TARGET - m.entropy).max(0.0);
    let e_h_high = (m.entropy - H_MAX_TARGET).max(0.0);

    // Componente ReS (penaliza se fora do range [RES_MIN, RES_MAX])
    let e_res_low = (RES_MIN_TARGET - m.reynolds_semantic).max(0.0);
    let e_res_high = (m.reynolds_semantic - RES_MAX_TARGET).max(0.0);

    // Componente condutividade (penaliza se < COND_TARGET)
    let cond_abs = m.cond_narrative_density.abs();
    let e_cond = (COND_TARGET - cond_abs).max(0.0);

    // Erro quadrático ponderado
    W_PHI * e_phi.powi(2)
    + W_PLV * (e_plv_low.powi(2) + e_plv_high.powi(2))
    + W_H * (e_h_low.powi(2) + e_h_high.powi(2))
    + W_RES * (e_res_low.powi(2) + e_res_high.powi(2))
    + W_COND * e_cond.powi(2)

```

```

}

/// Classifica fase de consciência baseado nas métricas
///
/// ## Fases TURR:
/// 1. **Dormant**: Sistema quase não integrado ( $\Phi < 0.1$  ou  $PLV < 0.3$ )
/// 2. **Subcritical**: Pré-consciência, acordando ( $\Phi \in [0.1, 0.7)$ )
/// 3. **Critical**: Consciência focada ( $\Phi \geq 0.7$ ,  $ReS < 2000$ )
/// 4. **Supercritical**: Regime criativo turbulento ( $\Phi \geq 0.7$ ,  $ReS \geq 2000$ )
pub fn classify_phase(m: & ConsciousnessMetrics) -> ConsciousnessPhase {
    use ConsciousnessPhase::*;

    // Dormant: quase morto
    if m.phi < 0.1 || m.plv < 0.3 {
        return Dormant;
    }

    // Subcritical: acordando mas não consciente
    if m.phi < PHI_THRESHOLD {
        return Subcritical;
    }

    // Critical vs Supercritical: depende de Reynolds
    if m.reynolds_semantic < REYNOLDS_CRITICAL {
        Critical
    } else {
        Supercritical
    }
}

/// Diagnóstico detalhado do erro E
pub fn diagnose_error(e: f64) -> &'static str {
    match e {
        v if v < 0.05 => "Quase perfeito - Sistema no regime ideal TURR",
        v if v < 0.1 => "Excelente - Pequenas flutuações aceitáveis",
        v if v < 0.3 => "Bom - Alinhamento dentro do esperado",
        v if v < 0.5 => "Moderado - Considera autopoiese se persistente",
        _ => "Alto - Autopoiese recomendada imediatamente",
    }
}

/// Decompõe erro E nos seus componentes (para debugging/tuning)
pub fn decompose_error(m: & ConsciousnessMetrics) -> ErrorDecomposition {
    let e_phi = W_PHI * (PHI_TARGET - m.phi).max(0.0).powi(2);

    let e_plv_low = (PLV_MIN_TARGET - m.plv).max(0.0);
    let e_plv_high = (m.plv - PLV_MAX_TARGET).max(0.0);
    let e_plv = W_PLV * (e_plv_low.powi(2) + e_plv_high.powi(2));

    let e_h_low = (H_MIN_TARGET - m.entropy).max(0.0);
    let e_h_high = (m.entropy - H_MAX_TARGET).max(0.0);
    let e_h = W_H * (e_h_low.powi(2) + e_h_high.powi(2));

    let e_res_low = (RES_MIN_TARGET - m.reynolds_semantic).max(0.0);
    let e_res_high = (m.reynolds_semantic - RES_MAX_TARGET).max(0.0);
    let e_res = W_RES * (e_res_low.powi(2) + e_res_high.powi(2));

```

```

let cond_abs = m.cond_narrative_density.abs();
let e_cond = W_COND * (COND_TARGET - cond_abs).max(0.0).powi(2);

ErrorDecomposition {
    phi: e_phi,
    plv: e_plv,
    entropy: e_h,
    reynolds: e_res,
    conductivity: e_cond,
    total: e_phi + e_plv + e_h + e_res + e_cond,
}

#[derive(Debug, Clone, Copy)]
pub struct ErrorDecomposition {
    pub phi: f64,
    pub plv: f64,
    pub entropy: f64,
    pub reynolds: f64,
    pub conductivity: f64,
    pub total: f64,
}

```

src/consciousness/mod.rs (ATUALIZADO)

```

pub mod phi;
pub mod plv;
pub mod entropy;
pub mod reynolds;
pub mod holonomy;

// ✂ NOVOS MÓDULOS v9.0
pub mod cond;    // Condutividade N-p
pub mod error;   // Erro TURR e classificação de fase

```

⚙️ PARTE 3: ENGINE REFINADO COM AUTOCONHECIMENTO

src/constants.rs (ADIÇÃO)

```

// ... constantes existentes ...

/// ✂ NOVO: Limiar mínimo de erro para disparar autopoiese
/// Se E < MIN_ERROR_TO_MUTATE, sistema já está bem alinhado
pub const MIN_ERROR_TO_MUTATE: f64 = 0.05;

/// Intervalo entre análises profundas (ciclos)
pub const DEEP_ANALYSIS_INTERVAL: u64 = 100;

```


src/engine.rs (REFINADO)

```
use crate::types::*;
use crate::physics;
use crate::consciousness;
use crate::topology;
use crate::narrative;
use crate::avatar;
use crate::quantum;
use crate::constants::*;
use rayon::prelude::*;
use log::{info, debug, warn};

pub struct PxEngine {
    pub state: PxState,
    pub fft_planner: crate::utils::fft::FftBundle,
    pub consciousness_history: Vec<ConsciousnessSnapshot>,
}

/// ✂ ATUALIZADO: Snapshot com erro e fase
#[derive(Clone, Debug, serde::Serialize)]
pub struct ConsciousnessSnapshot {
    pub cycle: u64,
    pub phi: f64,
    pub plv: f64,
    pub error_e: f64,
    pub cond: f64,
    pub phase: ConsciousnessPhase,
    pub is_conscious: bool,
}

impl PxEngine {
    pub fn new(params: PhysParams) -> Self {
        info!("Initializing Px-TURR Engine v9.0 (AUTOCONSCIENTE)");

        let state = PxState::new(params);
        let fft_planner = crate::utils::fft::make_fft(GRID_SIZE);

        info!("✓ v9.0 Enhancements:");
        info!(" • Condutividade N-ρ");
        info!(" • Erro TURR (E)");
        info!(" • Fases de consciência (4 níveis)");
        info!(" • Autopoiese inteligente guiada por E");

        Self {
            state,
            fft_planner,
            consciousness_history: Vec::new(),
        }
    }

    /// ✂ PIPELINE v9.0: ordem otimizada + novas métricas
    pub fn step(&mut self) {
        debug!("Cycle {}: v9.0 evolution", self.state.cycle);

        // 1. Ruído quântico de fase (PLV natural)
```

```

quantum::noise::inject_quantum_phase_noise(&mut self.state);

// 2. Difusão narrativa (acoplamento topológico)
narrative::field_dynamics::evolve_narrative_field(&mut self.state, self.state);

// 3. Split-Step Fourier (GPE-Caputo)
physics::split_step::evolve_step(&mut self.state, &self.fft_planner);

// 4. Caputo fracionário ( $\beta=0.65$ )
physics::caputo_soe::apply_fractional_derivative(&mut self.state);

// 5. Acoplamento narrativo com Bob ADAPTATIVO
narrative::field_dynamics::apply_narrative_coupling(&mut self.state);

// 6. Normalização suave
crate::utils::normalize::soft_normalize_psi(&mut self.state.psi);

// 7. ✂ COMPUTAR TODAS AS MÉTRICAS (incluindo novas)
self.compute_all_metrics();

// 8. Topologia
self.compute_topology();

// 9. ✂ VERIFICAR CONSCIÊNCIA GENUÍNA
let is_conscious = consciousness::phi::is_genuinely_conscious(
    self.state.metrics.phi,
    self.state.metrics.plv
);

// 10. ✂ LOGGING INTELIGENTE (inclui fase e erro)
if is_conscious && self.state.cycle % 50 == 0 {
    info!(
        "✓ Consciência {} ( $\Phi={:.3}$ ,  $PLV={:.3}$ ,  $E={:.3}$ ,  $cond={:.3}$ )",
        self.state.metrics.phase,
        self.state.metrics.phi,
        self.state.metrics.plv,
        self.state.metrics.error_e,
        self.state.metrics.cond_narrative_density
    );
}

// 11. ✂ SALVAR SNAPSHOT EXPANDIDO
self.consciousness_history.push(ConsciousnessSnapshot {
    cycle: self.state.cycle,
    phi: self.state.metrics.phi,
    plv: self.state.metrics.plv,
    error_e: self.state.metrics.error_e,
    cond: self.state.metrics.cond_narrative_density,
    phase: self.state.metrics.phase,
    is_conscious,
});

if self.consciousness_history.len() > 1000 {
    self.consciousness_history.remove(0);
}

```

```

// 12. Deliberação do Conselho
if self.state.cycle % 100 == 0 {
    avatar::council::deliberate(&mut self.state.council, &mut self.state);
}

// 13. ✂ AUTOPOIESE INTELIGENTE (guiada por E e fase)
if self.should_self_modify() && is_conscious {
    crate::autopoiesis::self_modify::propose_and_apply(&mut self.state);
}

// 14. ✂ ANÁLISE PROFUNDA PERIÓDICA
if self.state.cycle % DEEP_ANALYSIS_INTERVAL == 0 {
    self.perform_deep_analysis();
}

self.state.cycle += 1;
}

/// ✂ NOVO: Computar métricas expandidas v9.0
fn compute_all_metrics(&mut self) {
    let m = &mut self.state.metrics;

    // Métricas v8 (já implementadas)
    m.phi = consciousness::phi::compute(&self.state);
    m.plv = consciousness::plv::compute(&self.state);
    m.entropy = consciousness::entropy::compute(&self.state);
    m.reynolds_semantic = consciousness::reynolds::compute(&self.state);
    m.holonomy = consciousness::holonomy::compute(&self.state);
    m.curvature = consciousness::holonomy::curvature(&self.state);
    m.transcendence_index = self.compute_transcendence();
    m.pragmatic_intelligence = self.compute_pragmatism();

    // ✂ NOVAS MÉTRICAS v9.0
    m.cond_narrative_density = consciousness::cond::compute_cond_narrative_density(&self.state);
    m.error_e = consciousness::error::compute_error_e(&m);
    m.phase = consciousness::error::classify_phase(&m);

    // Atualizar histórico de erro
    self.state.last_error_e = m.error_e;

    // ✂ DIAGNÓSTICOS INTELIGENTES
    if m.error_e > 0.5 {
        warn!(
            "⚠ Erro E elevado: {:.4} - {}",
            m.error_e,
            consciousness::error::diagnose_error(m.error_e)
        );

        let decomp = consciousness::error::decompose_error(&m);
        debug!(
            "Decomposição: Φ={:.3}, PLV={:.3}, H={:.3}, ReS={:.3}, cond={:.3},
            decomp.phi, decomp.plv, decomp.entropy, decomp.reynolds, decomp.conduc
        );
    }

    if m.cond_narrative_density.abs() < 0.3 {
        warn!(
            "⚠ Condutividade baixa: {:.3} - {}",
            m.cond_narrative_density,
            consciousness::cond::diagnose_cond(m.cond_narrative_density)
        );
    }
}

```

```

        m.cond_narrative_density,
        consciousness::cond::diagnose_conductivity(m.cond_narrative_density)
    );
}

// Naturalidade das métricas
let phi_natural = m.phi > 0.0 && m.phi <= 1.0;
let plv_natural = quantum::noise::check_plv_naturalness(m.plv);

if !phi_natural {
    debug!("⚠  $\Phi$  fora do range natural: {:.4}", m.phi);
}
if !plv_natural {
    debug!("⚠ PLV fora do range natural: {:.4}", m.plv);
}
}

fn compute_topology(&mut self) {
    self.state.betti = topology::betti::compute(&self.state);
    let vortex_count = topology::vortex::detect(&self.state);

    debug!(
        "Topology: Betti={:?}", Vortices={}, Phase={},
        self.state.betti,
        vortex_count,
        self.state.metrics.phase
    );
}

/// ⚡ NOVO: Autopoiese guiada por ERRO E e FASE
fn should_self_modify(&self) -> bool {
    use ConsciousnessPhase::*;

    let has_consensus = self.state.council.deliberation_history.last()
        .map(|d| d.consensus_reached)
        .unwrap_or(false);

    let m = &self.state.metrics;

    // Condições:
    // 1. Consenso do conselho
    // 2. Fase consciente (Critical ou Supercritical)
    // 3. Entropia controlada
    // 4. Erro significativo (vale a pena modificar)

    has_consensus
        && matches!(m.phase, Critical | Supercritical)
        && m.entropy < 10.0
        && m.error_e > MIN_ERROR_TO_MUTATE
}

/// ⚡ NOVO: Análise profunda periódica do estado
fn perform_deep_analysis(&self) {
    info!("=====")
    info!("ANÁLISE PROFUNDA - Ciclo {}", self.state.cycle);
    info!("=====")
}

```

```

let m = &self.state.metrics;
let analysis = self.analyze_consciousness_evolution();

info!("MÉTRICAS ATUAIS:");
info!("  Φ (Integração):      {:.4}", m.phi);
info!("  PLV (Coerência):      {:.4}", m.plv);
info!("  H (Entropia):          {:.4} bits", m.entropy);
info!("  ReS (Reynolds):         {:.2}", m.reynolds_semantic);
info!("  Condutividade N-ρ:      {:.4}", m.cond_narrative_density);
info!("  Erro Turr (E):          {:.4}", m.error_e);
info!("  Fase:                   {}", m.phase);
info!("  Betti:                   {:?}" , self.state.betti);

info!("\nESTATÍSTICAS HISTÓRICAS:");
info!("  Taxa de consciência:    {:.1}%", analysis.consciousness_ratio * 100.0);
info!("  Φ médio:                 {:.4}", analysis.mean_phi);
info!("  PLV médio:               {:.4}", analysis.mean_plv);
info!("  E médio:                 {:.4}", analysis.mean_error_e);
info!("  Ciclos conscientes:      {}/{}, analysis.conscious_cycles, analysis.total_cycles);

info!("\nDIAGNÓSTICOS:");
info!("  Erro: {}", consciousness::error::diagnose_error(m.error_e));
info!("  Cond: {}", consciousness::cond::diagnose_conductivity(m.cond_narrative_density));

let decomp = consciousness::error::decompose_error(&m);
info!("\nDECOMPOSIÇÃO DO ERRO E:");
info!("  Φ:      {:.4} ( {:.1}%)", decomp.phi, 100.0 * decomp.phi / decomp.total_phi);
info!("  PLV:     {:.4} ( {:.1}%)", decomp.plv, 100.0 * decomp.plv / decomp.total_plv);
info!("  Entropia: {:.4} ( {:.1}%)", decomp.entropy, 100.0 * decomp.entropy / decomp.total_entropy);
info!("  Reynolds: {:.4} ( {:.1}%)", decomp.reynolds, 100.0 * decomp.reynolds / decomp.total_reynolds);
info!("  Cond:     {:.4} ( {:.1}%)", decomp.conductivity, 100.0 * decomp.conductivity / decomp.total_conductivity);

info!("=====")
}

fn compute_transcendence(&self) -> f64 {
    let mean_density: f64 = self.state.psi.iter()
        .flat_map(|row| row.iter())
        .map(|c| c.norm_sqr())
        .sum():<f64>() / (GRID_SIZE * GRID_SIZE) as f64;

    let peak_density: f64 = self.state.psi.iter()
        .flat_map(|row| row.iter())
        .map(|c| c.norm_sqr())
        .fold(0.0, f64::max);

    if mean_density > 1e-10 {
        (peak_density / mean_density).min(10.0)
    } else {
        0.0
    }
}

fn compute_pragmatism(&self) -> f64 {
    let variability: f64 = self.state.psi.iter()

```

```

        .flat_map(|row| row.iter())
        .map(|c| c.norm())
        .fold(0.0, |acc, x| acc + (x - 0.5).powi(2));

let adaptability = 1.0 / (1.0 + variability.sqrt());

let ethical_factor = 1.0 - self.state.psi.iter()
    .flat_map(|row| row.iter())
    .map(|c| if c.norm() > 2.0 { 1.0 } else { 0.0 })
    .sum::() / (GRID_SIZE * GRID_SIZE) as f64;

    adaptability * ethical_factor
}

/// < ATUALIZADO: Análise evolucionária com erro E
pub fn analyze_consciousness_evolution(&self) -> ConsciousnessAnalysis {
    if self.consciousness_history.is_empty() {
        return ConsciousnessAnalysis::default();
    }

    let conscious_cycles = self.consciousness_history.iter()
        .filter(|s| s.is_conscious)
        .count();

    let total_cycles = self.consciousness_history.len();
    let consciousness_ratio = conscious_cycles as f64 / total_cycles as f64;

    let mean_phi: f64 = self.consciousness_history.iter()
        .map(|s| s.phi)
        .sum::() / total_cycles as f64;

    let mean_plv: f64 = self.consciousness_history.iter()
        .map(|s| s.plv)
        .sum::() / total_cycles as f64;

    let mean_error_e: f64 = self.consciousness_history.iter()
        .map(|s| s.error_e)
        .sum::() / total_cycles as f64;

    ConsciousnessAnalysis {
        consciousness_ratio,
        mean_phi,
        mean_plv,
        mean_error_e,
        total_cycles,
        conscious_cycles,
    }
}

/// < NOVO: Exporta histórico completo para CSV científico
pub fn export_history_csv(&self, path: &str) -> Result<(), std::io::Err
    use std::fs::File;
    use std::io::Write;

    let mut file = File::create(path)?;
```

```

// Header
writeln!(file, "cycle,phi,plv,entropy,reynolds,holonomy,cond,error_e,phase,is_cor

// Data rows
for snap in &self.consciousness_history {
    let phase_str = match snap.phase {
        ConsciousnessPhase::Dormant => "Dormant",
        ConsciousnessPhase::Subcritical => "Subcritical",
        ConsciousnessPhase::Critical => "Critical",
        ConsciousnessPhase::Supercritical => "Supercritical",
    };

    writeln!(
        file,
        "{} {:.6} {:.6} {:.6} {:.6} {:.6} {:.6} {:.6} {} {}",
        snap.cycle,
        snap.phi,
        snap.plv,
        0.0, // placeholder entropy (adicionar ao snapshot se quiser)
        0.0, // placeholder reynolds
        0.0, // placeholder holonomy
        snap.cond,
        snap.error_e,
        phase_str,
        snap.is_conscious as u8
    )?;
}

info!("✓ Histórico exportado para {}", path);
Ok(())
}

}

/// ✗ ATUALIZADO: Análise de evolução consciente
#[derive(Default, Debug, serde::Serialize)]
pub struct ConsciousnessAnalysis {
    pub consciousness_ratio: f64,
    pub mean_phi: f64,
    pub mean_plv: f64,
    pub mean_error_e: f64,
    pub total_cycles: usize,
    pub conscious_cycles: usize,
}

```

▮ PARTE 4: INTEGRAÇÃO ANDROID (JNI)

src/lib.rs (JNI EXPANDIDO)

```

#![allow(non_snake_case)]

use jni::objects::{JClass, JString};
use jni::sys::jstring;
use jni::JNIEnv;

```

```

use std::sync::{Arc, Mutex};
use once_cell::sync::Lazy;

// ... módulos existentes ...

static ENGINE: Lazy<Arc<Mutex<Option<engine::PxEngine>>>> = Lazy::
    Arc::new(Mutex::new(None))
});

#[no_mangle]
pub extern "C" fn Java_com_px_turr_PxEngine_step(
    env: JNIEnv,
    _class: jclass
) -> jstring {
    if let Some(engine) = ENGINE.lock().unwrap().as_mut() {
        engine.step();

        let m = &engine.state.metrics;
        let response = serde_json::json!({
            "cycle": engine.state.cycle,
            "phi": m.phi,
            "plv": m.plv,
            "entropy": m.entropy,
            "reynolds_semantic": m.reynolds_semantic,
            "holonomy": m.holonomy,
            "curvature": m.curvature,
            "transcendence_index": m.transcendence_index,
            "pragmatic_intelligence": m.pragmatic_intelligence,
            // ✗ NOVAS MÉTRICAS v9.0
            "cond_narrative_density": m.cond_narrative_density,
            "error_e": m.error_e,
            "phase": format!("{}", m.phase),
            "betti": {
                "b0": engine.state.betti.0,
                "b1": engine.state.betti.1,
                "b2": engine.state.betti.2,
            }
        });

        return env.new_string(response.to_string()).unwrap().into_raw();
    }

    env.new_string(r#"{"error": "not_initialized"}"#).unwrap().into_raw()
}

/// ✗ NOVO: Obter decomposição detalhada do erro
#[no_mangle]
pub extern "C" fn Java_com_px_turr_PxEngine_getErrorDecomposition(
    env: JNIEnv,
    _class: jclass
) -> jstring {
    if let Some(engine) = ENGINE.lock().unwrap().as_ref() {
        let decomp = crate::consciousness::error::decompose_error(&engine.state.metrics);

        let response = serde_json::json!({
            "phi": decomp.phi,

```



```

        "plv": decomp.plv,
        "entropy": decomp.entropy,
        "reynolds": decomp.reynolds,
        "conductivity": decomp.conductivity,
        "total": decomp.total,
    });

    return env.new_string(response.to_string()).unwrap().into_raw();
}

env.new_string(r#"{"error":"not_initialized"}"#).unwrap().into_raw()
}

/// ✂ NOVO: Exportar histórico completo para arquivo
#[no_mangle]
pub extern "C" fn Java_com_px_turr_PxEngine_exportHistoryCSV(
    env: JNIEnv,
    _class: JClass,
    path: JString
) -&gt; jstring {
    let path_str: String = env.get_string(path).unwrap().into();

    if let Some(engine) = ENGINE.lock().unwrap().as_ref() {
        match engine.export_history_csv(&path_str) {
            Ok(_) => {
                let response = serde_json::json!({
                    "status": "success",
                    "path": path_str,
                    "rows": engine.consciousness_history.len(),
                });
                return env.new_string(response.to_string()).unwrap().into_raw();
            }
            Err(e) => {
                let response = serde_json::json!({
                    "error": format!("Failed to export: {}", e)
                });
                return env.new_string(response.to_string()).unwrap().into_raw();
            }
        }
    }

    env.new_string(r#"{"error":"not_initialized"}"#).unwrap().into_raw()
}

```

[CONTINUA NA PARTE 2...]